

Metodología de Sistemas II

Trabajo Práctico 3: Clean Code / Code Review

Farias, Gustavo

Matrícula: 101662

Comisión: M2025-13

Repositorio GitHub: [Lucenear/Tecnicatura_Programacion_UTN](https://github.com/Lucenear/Tecnicatura_Programacion_UTN)

Parte A - Aplicación de Clean Code

Aplicando los principios de Clean Code al código original de Facturacion para mejorar su calidad interna, encuentro lo siguiente:

1. Nombres de clases, métodos y variables (Legibilidad)

Problema original: El método calc y las variables p, o, t, r, e no revelan su intención y hay que estar adivinando qué significan.

Mejora aplicada:

```
calc -> → calcularTotalFacturacion  
p -> esPacienteParticular  
o -> tieneObraSocial  
t -> cantidadTurnosPrevios  
r -> totalAcumulado  
e -> precioEstudio
```

Impacto: El código ahora se lee fácilmente, eliminando la necesidad de comentarios explicativos.

2. Separación de responsabilidades y Cohesión interna

Problema original: El método único mezclaba varias responsabilidades porque realizaba iteración, matemáticas, evaluación de reglas de negocio particulares, de obra social y de fidelidad.

Mejora aplicada: Se divide la lógica en bloques cohesivos. La clase Facturacion orquesta el proceso, delegando el cálculo base, el descuento por cobertura y el descuento por fidelidad a secciones lógicas claramente delimitadas. Se extraen constantes para los numeros hardcodeados (0.5, 0.9, 3, 100), agrupando las reglas de negocio en la parte superior de la clase.

3. Reducción de acoplamiento

Problema original: El método recibía 4 parámetros primitivos sueltos. Cualquier clase que quisiera usar Facturacion estaba obligada a conocer el orden y tipo exacto de esos 4 datos.

Mejora aplicada: Se desacopla la lógica de los numeros hardcodeados del flujo de control. Si el hospital cambia el descuento del 10% al 15%, solo se toca una constante, no el flujo lógico.

4. Manejo de errores

Problema original: Si estudios era null, el código original lanzaba un NullPointerException en el for. Si los valores eran negativos, generaba facturas con montos absurdos sin avisar.

Mejora aplicada: Se agrega validación al inicio del método. Se verifica que la lista no sea nula y se lanza una excepción controlada con un mensaje claro.

5. Legibilidad general

Se eliminaron condicionales redundantes como if (p == true) reemplazándolos por if (esPacienteParticular).

6. Código Facturacion

```
import java.util.List;

public class Facturacion {

    // Constantes para eliminar hardcodeo de valores
    private static final double FACTOR_PARTICULAR = 0.5;
    private static final double FACTOR_OBRA_SOCIAL = 0.9;
    private static final int UMBRAL_TURNOS_DESCUENTO = 3;
    private static final double MONTO_DESCUENTO_FIJO = 100.0;

    public double calcularTotalFacturacion(List<Double>
preciosEstudios,
                                         boolean
esPacienteParticular,
                                         boolean
tieneObraSocial,
                                         int
cantidadTurnosPrevios) {

        // Manejo de errores
        if (preciosEstudios == null) {
            throw new IllegalArgumentException("La lista de
precios de estudios no puede ser nula.");
        }

        // Separacion visual de responsabilidades mediante bloques
        logicos
        double totalAcumulado = 0.0;
```

```

        // Calculo base segun tipo de paciente
        for (Double precioEstudio : preciosEstudios) {
            if (esPacienteParticular) { // Elimino el redundante
                totalAcumulado += precioEstudio *
                FACTOR_PARTICULAR;
            } else {
                totalAcumulado += precioEstudio;
            }
        }

        // Aplicacion de descuento por Obra Social
        if (tieneObraSocial) {
            totalAcumulado = totalAcumulado * FACTOR_OBRA_SOCIAL;
        }

        // Aplicacion de descuento por volumen de turnos
        if (cantidadTurnosPrevios > UMBRAL_TURNOS_DESCUENTO) {
            totalAcumulado = totalAcumulado -
            MONTO_DESCUENTO_FIJO;
        }

        return totalAcumulado;
    }
}

```

Parte B - Refactoring Seguro

Sobre el código mejorado en la Parte A, se aplicaron formalmente las siguientes técnicas de refactorización:

Técnica Aplicada	Problema que resolvía	Mejora que aporta al diseño
Extraer metodo	El método principal era largo y hacía múltiples cosas (iterar, calcular base, aplicar descuento 1, aplicar descuento 2).	Divide el algoritmo complejo en métodos pequeños con nombres descriptivos (`calcularSubtotal`, `aplicarDescuentoCobertura`, etc.). Facilita el testing unitario de cada regla de negocio por separado y mejora drásticamente la legibilidad.

Introducir parametro de objeto	El método recibía una lista larga de parámetros primitivos (`List`, `boolean`, `boolean`, `int`). Era fácil invocar el método pasando los argumentos en el orden equivocado.	Agrupar los datos que viajan juntos en una nueva clase (`DetalleFacturacion`). Reduce la signatura del método, mejora la cohesión de los datos y permite agregar nuevos parámetros en el futuro sin romper las llamadas existentes.
Mover metodo	La validación de los datos de entrada (manejo de errores agregado en la Parte A) estaba dentro de la clase de servicio `Facturacion`, pero operaba puramente sobre los datos crudos.	Se movió la lógica de validación (`validar()`) hacia la propia clase de datos `DetalleFacturacion`. Esto acerca el comportamiento a los datos (Tell, Don't Ask), aumentando la cohesión del objeto de datos y liberando a `Facturacion` de responsabilidades que no le corresponden.

DetalleFacturacion.java:

```
import java.util.List;

//Resolvemos el problema de la lista larga de parametros
primitivos en Facturacion
public class DetalleFacturacion {
    private final List<Double> preciosEstudios;
    private final boolean esPacienteParticular;
    private final boolean tieneObraSocial;
    private final int cantidadTurnosPrevios;

    public DetalleFacturacion(List<Double> preciosEstudios,
boolean esPacienteParticular,
boolean tieneObraSocial, int
cantidadTurnosPrevios) {
        this.preciosEstudios = preciosEstudios;
        this.esPacienteParticular = esPacienteParticular;
        this.tieneObraSocial = tieneObraSocial;
        this.cantidadTurnosPrevios = cantidadTurnosPrevios;
    }
}
```

```

// Se movio la validacion desde Facturacion hacia aqui
public void validar() {
    if (preciosEstudios == null) {
        throw new IllegalArgumentException("La lista de
precios de estudios no puede ser nula.");
    }
}

// Getters necesarios para que Facturacion acceda a los datos
public List<Double> getPreciosEstudios() { return
preciosEstudios; }
public boolean isEsPacienteParticular() { return
esPacienteParticular; }
public boolean isTieneObraSocial() { return tieneObraSocial; }
public int getCantidadTurnosPrevios() { return
cantidadTurnosPrevios; }
}

```

Facturacion.java

```

public class Facturacion {

    private static final double FACTOR_PARTICULAR = 0.5;
    private static final double FACTOR_OBRA_SOCIAL = 0.9;
    private static final int UMBRAL_TURNOS_DESCUENTO = 3;
    private static final double MONTO_DESCUENTO_FIJO = 100.0;

    public double calcularTotalFacturacion(DetalleFacturacion
detalle) {
        // Uso el metodo movido para validar
        detalle.validar();

        // El metodo principal ahora es legible
        double subtotal = calcularSubtotalEstudios(detalle);
        double totalConCobertura =
aplicarDescuentoCobertura(subtotal, detalle.isTieneObraSocial());
        double totalFinal =
aplicarDescuentoFidelidad(totalConCobertura,
detalle.getCantidadTurnosPrevios());

        return totalFinal;
    }
}

```

```

        // Extrae la logica de iteracion y colculo base
        private double calcularSubtotalEstudios(DetalleFacturacion
detalle) {
            double totalAcumulado = 0.0;
            for (Double precioEstudio : detalle.getPreciosEstudios())
            {
                if (detalle.isEsPacienteParticular()) {
                    totalAcumulado += precioEstudio *
FACTOR_PARTICULAR;
                } else {
                    totalAcumulado += precioEstudio;
                }
            }
            return totalAcumulado;
        }

        // Extrae la regla de negocio de la obra social
        private double aplicarDescuentoCobertura(double monto, boolean
tieneObraSocial) {
            if (tieneObraSocial) {
                return monto * FACTOR_OBRA_SOCIAL;
            }
            return monto;
        }

        // Extrae la regla de negocio de fidelidad
        private double aplicarDescuentoFidelidad(double monto, int
cantidadTurnos) {
            if (cantidadTurnos > UMBRAL_TURNOS_DESCUENTO) {
                return monto - MONTO_DESCUENTO_FIJO;
            }
            return monto;
        }
    }
}

```

Main.java

```

import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        // Simulo los datos que llegarían al sistema
        List<Double> estudiosRealizados = Arrays.asList(1000.0,

```

```

2000.0, 500.0);

    // Creo el objeto parametro (Antes eran 4 argumentos
    // Paciente Particular (true), Con Obra Social (true), 4
    // Turnos previos
    DetalleFacturacion detalle = new
    DetalleFacturacion(estudiosRealizados, true, true, 4);

    Facturacion sistema = new Facturacion();

    try {
        double total =
sistema.calcularTotalFacturacion(detalle);
        System.out.println("Total a facturar: $" + total);
    } catch (IllegalArgumentException e) {
        System.err.println("Error de facturación: " +
e.getMessage());
    }
}
}

```

Parte C - Checklist de Code Review

Pregunta de Revisión	Código Original (`Facturacion.java`)	Código Refactorizado	Justificación / Evidencia
Los nombres son descriptivos?	No	Si	Se cambiaron `calc`, `p`, `o`, `t` por nombres que explican el dominio del hospital.
Las funciones poseen una única responsabilidad?	No	Si	Gracias a 'extraer metodo', cada método privado hace exactamente una cosa.
Existe duplicación innecesaria?	Parcial	No	Se centralizaron los factores de descuento en constantes, evitando dispersión.

El código es fácilmente extensible?	No	Si	Agregar un nuevo descuento implica crear un nuevo método privado y llamarlo, sin alterar la lógica existente.
Hay manejo correcto de errores?	No	Si	Se implementó 'Fail Fast' con <code>`IllegalArgumentExcepcion`</code> para entradas nulas.
Existe acoplamiento innecesario?	Alto	Bajo	Introducir parametro de objeto' eliminó la dependencia de parámetros primitivos sueltos.
La cohesión es adecuada?	Bajo	Alto	Los datos viajan juntos en un objeto, y los métodos de la clase trabajan sobre un mismo objetivo.
Se respetan principios SOLID?	No	Si	SRP (métodos extraídos), OCP (fácil extensión), DIP (depende de la abstracción del objeto parámetro).
El código resulta legible para otro desarrollador?	No	Si	Cualquier desarrollador del Hospital puede leer <code>`calcularTotalFacturacion`</code> y entender el flujo.

Parte D - Feedback Profesional

Para: Equipo de Desarrollo - Módulo Facturación

De: Gustavo Farias

Asunto: Revisión de Pull Request #42 - Lógica de Facturación

Hola equipo:

Complete la revisión del código correspondiente al módulo de facturación (Facturacion.java). El sistema actualmente logra calcular los montos correctamente según los casos de prueba manuales.

Sin embargo, para garantizar que el sistema sea mantenible a medida que el Hospital Central incorpore nuevas reglas de negocio, detecte áreas de mejora importantes relacionadas con Clean Code y diseño orientado a objetos:

- Legibilidad y Nomenclatura: El uso de variables de una sola letra (p, o, t, r) oculta la intención del código. En un dominio tan crítico como la facturación médica, necesitamos que el código se auto-documente. Recomendando renombrar estas variables para reflejar conceptos del negocio (por ejemplo, `esPacienteParticular`, `cantidadTurnosPrevios`).
- Numeros fijos: Tenemos valores como 0.5, 0.9 y 100 directamente en las operaciones matemáticas. Si el Ministerio de Salud ajusta los aranceles o los descuentos por obra social, modificar estos números dispersos en el código es propenso a errores. Sugiero extraerlos a constantes privadas descriptivas al inicio de la clase.
- Responsabilidad Única y Complejidad: El método `calc` actualmente maneja la iteración de estudios, la lógica de pacientes particulares, el descuento de obra social y el descuento por volumen de turnos. Esto dificulta su lectura y hace imposible testear cada regla de descuento de forma aislada. Aplicando la técnica de Extraer Metodo, podemos separar cada regla en su propio método, logrando que el método principal sea simplemente un resumen ejecutivo del proceso de facturación.
- Acoplamiento por Parámetros: Pasar cuatro parámetros independientes hace que la firma del método sea frágil. Mediante Introduccion de parametro objeto, podemos encapsular estos datos en un objeto `DetalleFacturacion`. Esto no solo limpia la firma del método, sino que nos permite aplicar `Mover metodo` para trasladar las validaciones de nulos directamente a este nuevo objeto, mejorando el manejo de errores.

He preparado una propuesta de refactorización aplicando estos cambios en la rama `refactor/clean-code-facturacion`. El comportamiento funcional se ha mantenido intacto (los resultados numéricos son idénticos), pero la estructura interna ahora es robusta y extensible.

Por favor, revisen los cambios y dejen sus comentarios.

Saludos!